

AI Infra · 多轮项目深挖

同一项目如何经得住从业务约束到 Kernel 细节的连续追问

项目深挖

数据结构

并行策略

验证边界

岗位能力画像

岗位画像：多轮面试不只增加题量，而是逐层验证项目所有权、设计取舍、正确性边界和系统视角。

高频深挖：题目背后的判断

项目怎样开场才可追问？

用对象、约束、现象、证据和结果开场，避免先堆名词。

问题 → 机制 → 方案 → 验收 → 边界。

Hash 一定比连续 buffer 好吗？

token/layer 等连续下标常可压成 dense tensor；比较 key 开销、locality、扩容和稀疏性。

数据结构要由访问模式决定。

并行概念怎么不混？

分别说明数据、张量、流水、专家并行在切什么，通信发生在哪里，瓶颈何时转移。

先画数据流，再说缩写。

量化 / fallback 如何讲？

先说误差和覆盖边界，再说高性能路径；未覆盖 shape 必须回退。

性能产品 = fast path + safe path + tests。

项目叙事的通用深挖链

问题与约束

证据与机制

方案与取舍

验证与边界

答题提醒

不要从‘用了什么库/指令’开始；先固定工作负载、指标和约束，再说明数据流/资源/依赖，最后给出正确性、性能和反例边界。

把一句结论变成可复验的事实

现象

固定 workload / 阶段 / 指标，明确‘慢’或‘错’在哪里。

证据

用 trace、counter、对照或最小复现排除相邻假设。

机制

说明资源、数据流或依赖如何导致这个现象。

验收

数值、边界、重复性和 E2E 共同成立后再下结论。

一题答透：五格回答法：每个项目都能被验证

问题 “请讲一个最近的优化；随后追问数据从哪来、为什么慢、为什么这样改。”

问题

对象是什么、目标指标是什么、约束是什么；避免把 kernel、operator、E2E 混成一个数字。

机制

画数据/控制流和关键资源，说明哪一步形成等待、无效计算或额外访存。

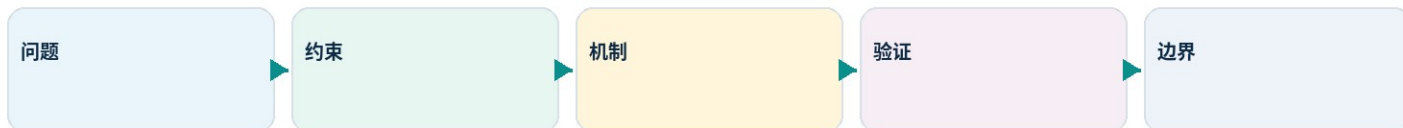
方案

给备选与取舍；说明 selector、fallback、状态机或通信协议如何落地。

验证

数值/确定性、性能、异常路径、多卡/动态 shape 与端到端分别验收。

把推理链画出来



常见误区

× 只背最终百分比，无法解释测量层级。

× 把团队 kernel 写成个人实现，或把接入写成底层算法创新。

× 没有说回退、失败路径和反例。

术语速懂 | 面试怎么答

术语速懂

Shape-aware selector

按 shape、dtype、layout、对齐和并行配置选择快路径或安全回退，是性能交付的一部分。

Dense index vs Hash

下标连续且密集时 dense tensor 通常少 key 计算与指针跳转；稀疏、动态时 hash 更合适。

面试中怎么答

① 先固定对象、指标、约束，避免把 kernel、operator 和 E2E 混成一个数字。

② 再说数据/控制流和备选方案的取舍，明确 fast path 条件。

③ 最后给数值、异常、多卡/动态 shape 与反例边界。

白板 / 手撕：白板题：二分答案 + 可行性检查

题面 把连续数组分为 k 段，使最大段和最小。请给出边界、check 函数和复杂度。

C++17 · binary search with feasibility check

核心实现

```
01 long long min_largest_sum(const vector<int>& a, int k) {
02     long long lo = *max_element(a.begin(), a.end());
03     long long hi = accumulate(a.begin(), a.end(), 0LL);
04     while (lo < hi) {
05         long long mid = lo + (hi - lo) / 2, sum = 0;
06         int parts = 1;
07         for (int x : a)
08             if (sum + x > mid) { ++parts; sum = x; }
09             else sum += x;
10         if (parts <= k) hi = mid; else lo = mid + 1;
11     }
12     return lo;
13 }
```

写代码时怎么讲

1. left = max(nums), right = sum(nums) : 答案落在此闭区间。
2. check(mid) : 从左到右贪心累加，超 mid 就开新段。
3. 可行条件是段数 <= k ; 二分收缩直到 left == right。
4. 讨论 k=1、k>=n、单元素、溢出和复杂度 $O(n \log \text{sum})$ 。

实现重点

- 前提：元素非负，check(mid) 才具有单调性。
- 复杂度： $O(n \log(\text{sum}))$ ，用 long long 避免溢出。
- 覆盖 k=1、k>=n、空输入和单元素。

边界、测试与追问

最少要说明的验证

- 前提：元素非负，check(mid) 才具有单调性。
- 复杂度： $O(n \log(\text{sum}))$ ，用 long long 避免溢出。
- 覆盖 k=1、k>=n、空输入和单元素。

面试继续追问

- 区间合并的排序不变量是什么？
- 二维 reduce 如何选择 block/warp partition？
- 项目里什么结果不能外推到 E2E？